

# Основы тестирования серверных приложений

Unit, integration и end-to-end tests, AAA, assertions и тестируемая архитектура

# Результаты занятия

- ✓ Объяснять назначение автоматических тестов backend
- ✓ Различать unit, integration и end-to-end tests
- ✓ Понимать test pyramid
- ✓ Формулировать test case по схеме Arrange–Act–Assert
- ✓ Использовать assertions
- ✓ Проверять позитивные, негативные и граничные сценарии
- ✓ Понимать test isolation и deterministic tests
- ✓ Отделять test environment от development/production
- ✓ Писать первый test через node:test
- ✓ Запускать tests через node --test

# Учебные вопросы

---

- 1** **Зачем тестировать backend** Регрессии, уверенность в изменениях и автоматическая проверка поведения
- 2** **Уровни тестирования** Unit, integration и end-to-end
- 3** **Структура хорошего теста** Arrange–Act–Assert, assertions и test cases
- 4** **Качество тестов** Isolation, determinism, boundary cases и test doubles
- 5** **Первый автоматический test** node:test, node:assert/strict и node --test



# Учебный вопрос 1. Зачем тестировать backend

Автоматический test проверяет ожидаемое поведение программы повторяемым способом

# Что такое программный test

---

- Test — автоматическая проверка, которая запускает часть программы с заданными входными данными и сравнивает фактический результат с ожидаемым.
- Test фиксирует ожидаемое поведение системы.
- Test можно запускать многократно.
- Успешный test не доказывает отсутствие всех ошибок.
- Набор tests уменьшает риск случайной регрессии.

# Регрессия

---

- Регрессия — ситуация, когда изменение кода ломает уже работавшее поведение.
- Добавили новое поле Product и сломали старый Controller.
- Изменили расчёт Order total и получили неверную сумму.
- Исправили Authorization и случайно закрыли публичный endpoint.
- Tests помогают обнаружить такие изменения до deployment.

# Manual testing и Automated testing

## Manual testing

- Человек вручную выполняет шаги
- Полезно для exploratory checks
- Медленно повторять после каждого изменения
- Легко пропустить старый сценарий

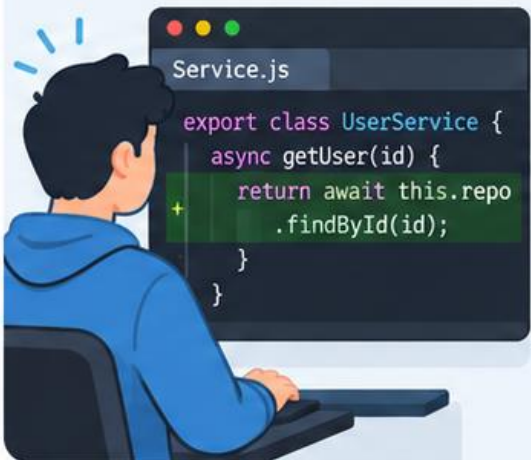
## Automated testing

- Сценарий описан кодом
- Запускается одной командой
- Повторяется одинаково
- Подходит для regression suite

# Изменение кода → test suite → pass/fail → решение о следующем шаге

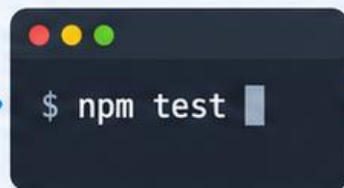
## 1. Разработчик изменяет код

Например, модуль Service



## 2. Запускает тесты

В терминале



## 3. Test suite выполняется

Запускаются все тесты проекта (юнит, интеграционные и др.)



### Test Suite

- ✓ UserService.test.js ✓
- ✓ AuthService.test.js ✓
- ✓ Repository.test.js ✓
- ✓ API.test.js ✓

...

5/5 tests

12.3s

### PASS

#### Тесты прошли успешно



### Tests passed

Все тесты успешно пройдены



### Продолжить разработку

Можно добавлять новый функционал



### Refactor

Безопасно улучшать код (рефакторинг)

### FAIL

#### Тесты не прошли



### Tests failed

Есть упавшие тесты



### Найти причину

Изучить вывод тестов, логи, стек ошибок



### Исправить код

Внести необходимые изменения в Service и др.



### Повторный запуск

Снова запустить тесты (npm test)

Вернуться к test suite

Пока все тесты не пройдут



Тесты подсказывают, что делать дальше после изменения кода:

если **PASS** — можно двигаться дальше, если **FAIL** — нужно найти причину, исправить код и запустить тесты снова.

# Что тестировать в Internet Shop

---

- Расчёт стоимости Order.
- Validation бизнес-правил.
- Product Service.
- Auth Service.
- REST endpoints.
- Authorization rules.
- Работу Prisma с test database.
- Полный путь запроса от HTTP до БД.

# Tests как executable specification

---

- Название test case описывает ожидаемое поведение.
- Input и expected result становятся частью документации.
- Новый разработчик может понять контракт функции по tests.
- Хороший test отвечает на вопрос: что должна делать система в этом сценарии?



# Учебный вопрос 2. Уровни тестирования

Разные tests проверяют систему на разных уровнях

# Unit test

---

- Unit test проверяет небольшую изолированную единицу поведения — обычно функцию, класс или service method.
- Запускается быстро.
- Не должен зависеть от реальной сети или production DB.
- Ошибка обычно легко локализуется.
- Подходит для бизнес-логики.

# Unit test: расчёт стоимости заказа

Запрос: Проверить функцию `calculateOrderTotal(items)`

## JAVASCRIPT

```
const items = [  
  { price: 10000, quantity: 2 },  
  { price: 5000, quantity: 1 }  
];
```

## Результат

Ожидаемый `total` = 25000

## Пояснение

- Test проверяет чистую бизнес-функцию без HTTP и PostgreSQL.

# Integration test

---

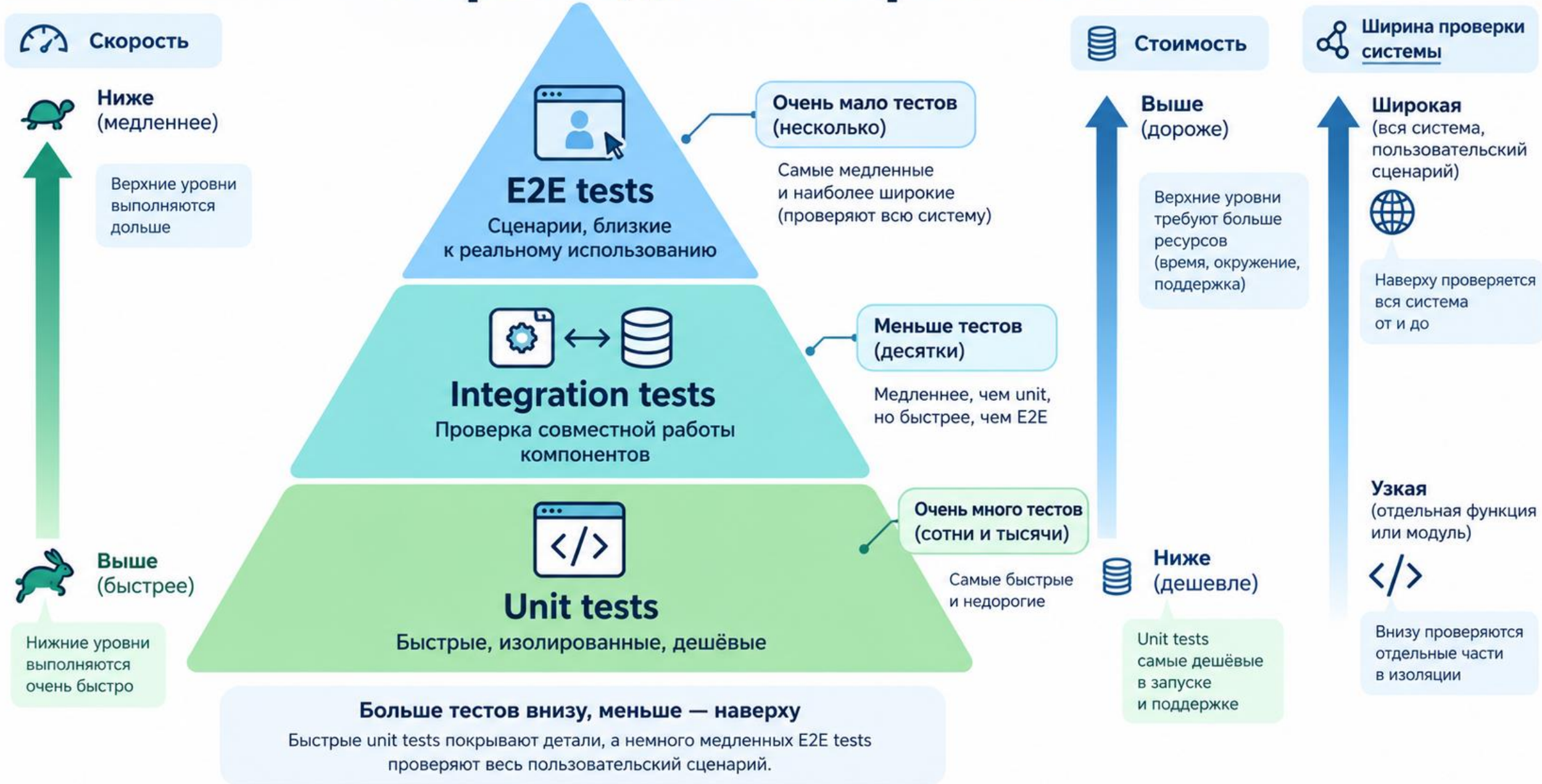
- Integration test проверяет совместную работу нескольких реальных компонентов.
- Express route + middleware + Controller.
- Service + Prisma + test database.
- JWT middleware + protected endpoint.
- Ошибку сложнее локализовать, чем в unit test.
- Такие tests обычно медленнее unit tests.

# End-to-end test

---

- End-to-end test проверяет систему максимально близко к реальному пользовательскому сценарию.
- Запуск приложения.
- HTTP request к API.
- Работа middleware, Service и DB.
- Проверка response.
- Самый широкий, но обычно самый дорогой по времени уровень.

# Пирамида тестирования



# Сравнение уровней тестирования

| Уровень     | Что проверяет         | Скорость | Типичный пример             |
|-------------|-----------------------|----------|-----------------------------|
| Unit        | Одна функция/модуль   | Высокая  | calculateOrderTotal()       |
| Integration | Несколько компонентов | Средняя  | Service + Prisma            |
| E2E         | Почти вся система     | Ниже     | POST /api/orders → DB → 201 |

# Почему не делать только E2E

---

- E2E test запускает много компонентов одновременно.
- Он медленнее.
- Причина failure может находиться в любом слое.
- Набор только из E2E становится дорогим и хрупким.
- Test pyramid помогает распределить проверки по подходящим уровням.

# Не каждую функцию нужно тестировать отдельно

---

- Тестируйте значимое поведение, а не каждую строку кода.
- Простые getter/setter обычно не требуют отдельного test.
- Приоритет — бизнес-правила, преобразования и опасные edge cases.
- Test должен давать полезный сигнал при failure.



## Учебный вопрос 3. Структура хорошего теста

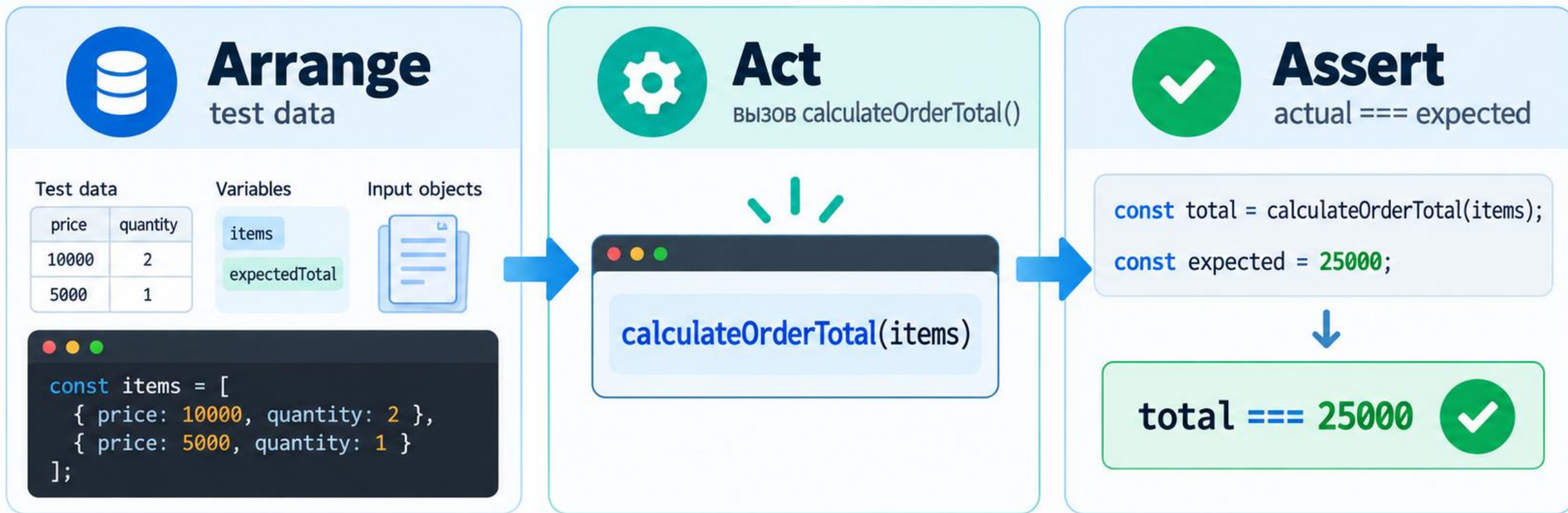
Test case должен быть понятным, коротким и воспроизводимым

# Arrange – Act – Assert

---

- Arrange — подготовить входные данные и окружение.
- Act — выполнить тестируемое действие.
- Assert — проверить результат.
- AAA делает test case читаемым и помогает отделять подготовку от проверки.

# AAA: Arrange → Act → Assert



**Структура хорошего теста:** сначала подготовка данных, затем действие, затем проверка результата.

# AAA на примере Order total

## JAVASCRIPT

```
test("calculates total for several items", () => {  
  // Arrange  
  const items = [  
    { price: 10000, quantity: 2 },  
    { price: 5000, quantity: 1 }  
  ];  
  
  // Act  
  const total = calculateOrderTotal(items);  
  
  // Assert  
  assert.strictEqual(total, 25000);  
});
```

## Что важно

- Arrange создаёт test data.
- Act вызывает только тестируемое поведение.
- Assert фиксирует ожидаемый результат.

# Assertion

---

- Assertion — проверка утверждения об ожидаемом состоянии программы.
- actual — фактический результат.
- expected — ожидаемый результат.
- Если условие не выполнено, test завершается failure.
- Node.js предоставляет стабильный модуль `node:assert`.

# Примеры assertions

| Assertion       | Назначение                  | Пример            |
|-----------------|-----------------------------|-------------------|
| strictEqual     | Строгое равенство           | total === 25000   |
| deepStrictEqual | Сравнение объектов/массивов | Product object    |
| ok              | Проверка truthy condition   | result.length > 0 |
| throws          | Ожидаемая sync error        | invalid quantity  |
| rejects         | Ожидаемая async rejection   | Service error     |

## Хорошее имя test case

---

- Название должно описывать наблюдаемое поведение.
- Хорошо: calculates total for several items.
- Хорошо: rejects quantity less than one.
- Плохо: test1.
- Плохо: order function works.

# Позитивные и негативные scenarios

---

- Positive test использует допустимые данные.
- Negative test проверяет некорректный input или запрещённое действие.
- Оба типа нужны.
- Только happy path оставляет ошибки на границах непроверенными.

# Test cases for a function

Проверяем функцию на разных типах входных данных

```
calculateOrderTotal(items, discount = 0)
```



## Happy path

Нормальный сценарий

Проверяем, что функция корректно работает с валидными данными.

```
const items = [
  { price: 100, quantity: 2 },
  { price: 50, quantity: 1 }
];
const discount = 10;

const total = calculateOrderTotal(items, discount);
```

Ожидаемый результат:

**230** (250 – 10% = 230)

Что проверяем:

- ✓ Корректный расчёт итоговой суммы
- ✓ Применение скидки
- ✓ Возврат правильного значения



## Invalid input

Ошибочный ввод

Проверяем, что функция корректно обрабатывает некорректные данные и выбрасывает ошибку.

```
const items = null;
const discount = 10;

calculateOrderTotal(items, discount);
```

Ожидаемый результат:

**Ошибка (throw)**

"Invalid items"

Что проверяем:

- ✓ Валидация входных данных
- ✓ Выбрасывание корректной ошибки
- ✓ Информативное сообщение об ошибке

## Одна функция — несколько направлений проверки

Комплексное тестирование помогает убедиться, что функция работает корректно в реальных и граничных ситуациях.



## Boundary values

Граничные значения

Проверяем, как функция работает на границах допустимых значений.

```
// Пустой заказ
const items = [];
const total = calculateOrderTotal(items, 0);

// Максимальная скидка
const items2 = [{ price: 100, quantity: 1 }];
const total2 = calculateOrderTotal(items2, 100);

// Граничное количество
const items3 = [{ price: 100, quantity: 0 }];
const total3 = calculateOrderTotal(items3, 0);
```

Ожидаемые результаты:

- Пустой заказ → 0
- Скидка 100% → 0
- Количество 0 → 0

Что проверяем:

- ✓ Корректная обработка граничных значений
- ✓ Отсутствие ошибок на границах
- ✓ Возврат ожидаемых результатов



Покрывая happy path, invalid input и boundary values, мы делаем функцию надёжной и устойчивой к реальным сценариям использования.

# Boundary values

---

- Ошибки часто возникают на границах диапазонов.
- Если quantity разрешён 1..100, проверяйте 1 и 100.
- Рядом с границей проверяйте 0 и 101.
- Если массив должен иметь минимум один элемент, отдельно проверьте пустой массив.



## Учебный вопрос 4. Качество тестов

Test должен падать только из-за проверяемого поведения

# Test isolation

---

- Один test не должен оставлять состояние для другого.
- Порядок запуска tests не должен менять результат.
- Shared mutable state создаёт случайные failures.
- База для integration tests должна очищаться или пересоздаваться.

# Deterministic test

---

- Deterministic test при одинаковых условиях всегда даёт одинаковый результат.
- Не зависит от текущей даты без контролируемого clock.
- Не зависит от случайного значения без фиксированной стратегии.
- Не зависит от внешнего production API.
- Не зависит от test execution order.

# Deterministic test

## Одинаковый input → одинаковый результат

Детерминированный тест при одинаковых входных данных всегда даёт один и тот же результат, независимо от времени запуска и окружения.



### Два независимых запуска одного теста

Одинаковые данные → одинаковый результат (PASS)

#### Запуск 1

🕒 10:00

```
import { calculateOrderTotal }
from './order.js';

test('total with discount', () => {
  const items = [
    { price: 100, quantity: 2 },
    { price: 50, quantity: 1 }
  ];
  const discount = 10;

  const result = calculateOrderTotal(
    items, discount);
  expect(result).toBe(230);
});
```



**PASS**

Результат: 230

Время выполнения: 12 мс

Одинаковый  
input



```
items = [
  { price: 100,
    quantity: 2 },
  { price: 50,
    quantity: 1 }
]

discount = 10
```



Одинаковый  
результат

#### Запуск 2

🕒 15:30

```
import { calculateOrderTotal }
from './order.js';

test('total with discount', () => {
  const items = [
    { price: 100, quantity: 2 },
    { price: 50, quantity: 1 }
  ];
  const discount = 10;

  const result = calculateOrderTotal(
    items, discount);
  expect(result).toBe(230);
});
```



**PASS**

Результат: 230

Время выполнения: 11 мс



## Что может сделать тест не детерминированным?

Эти зависимости могут приводить к разным результатам при каждом запуске.



### Текущее время (clock)

Результат зависит от текущей даты и времени.

```
const now = new Date();
expect(isExpired(now))
  .toBe(true);
```



### Сеть (network)

Зависимость от внешних сервисов и их доступности.

```
const res = await fetch(
  'https://api.example.com'
);
expect(res.status).toBe(200);
```



### Случайные значения (random)

Результат зависит от генератора случайных чисел.

```
const id = Math.random();
expect(id).toBeGreaterThan(
  0.5
);
```



### Общее состояние (shared state)

Результат зависит от состояния, оставленного предыдущими тестами.

```
let counter = 0;
test('inc', () => {
  counter++;
  expect(counter).toBe(1);
});
```



**Хороший тест использует только контролируемые и предсказуемые данные.**

При одинаковом input тест должен всегда давать одинаковый результат (PASS).

# Flaky test

---

- Flaky test иногда проходит, а иногда падает без изменения проверяемого кода.
- Причины: race conditions.
- Shared state.
- Реальная сеть.
- Timing assumptions.
- Неочищенная test database.

# Test doubles

---

- Stub возвращает заранее подготовленный результат.
- Mock позволяет проверять взаимодействие с dependency.
- Fake — упрощённая рабочая реализация dependency.
- Test doubles помогают изолировать unit от Prisma, email, filesystem и внешних API.
- Подробно mocks разберём в следующей лекции с Jest.

# Production dependency и Test double

## Production

- PostgreSQL
- SMTP server
- External payment API
- Filesystem storage

## Unit test

- Stub/Fake repository
- Mock mail service
- Fake payment client
- Temporary/fake storage

# Test environment

---

- Tests не должны использовать production database.
- Используйте отдельную DATABASE\_URL для test environment.
- Test data должна быть искусственной.
- Secrets production не нужны в tests.
- Для unit tests database часто вообще не требуется.



# Учебный вопрос 5. Первый автоматический test

Node.js содержит стабильный встроенный test runner

# node:test

---

- Встроенный test runner доступен через node:test.
- Начиная с Node.js 20 test runner имеет stable status.
- Запуск из CLI выполняется через node --test.
- Для базовых assertions можно использовать node:assert/strict.
- Вводная лабораторная не требует стороннего framework.

# Функция для тестирования

## JAVASCRIPT

```
export function calculateOrderTotal(items) {  
  return items.reduce(  
    (sum, item) => sum + item.price * item.quantity,  
    0  
  );  
}
```

# Первый test file

## JAVASCRIPT

```
import test from "node:test";
import assert from "node:assert/strict";
import { calculateOrderTotal } from "../services/orderMath.js";

test("calculates total for several items", () => {
  const items = [
    { price: 10000, quantity: 2 },
    { price: 5000, quantity: 1 }
  ];

  const total = calculateOrderTotal(items);

  assert.strictEqual(total, 25000);
});
```

## Что важно

- node:test объявляет test case.
- node:assert/strict предоставляет assertions.
- Test не запускает Express и PostgreSQL.
- Это классический unit test.

## Запуск tests

**BASH**

```
node --test
```

# Что означает pass/fail

---

- Pass — все assertions test case выполнены.
- Fail — хотя бы одна assertion не выполнена или test выбросил неожиданную error.
- Process exit code отличен от нуля при failing test suite.
- Это позволяет позже использовать tests в CI/CD pipeline.

# Negative test c assert.throws()

## JAVASCRIPT

```
test("rejects quantity less than one", () => {  
  const items = [  
    { price: 10000, quantity: 0 }  
  ];  
  
  assert.throws(  
    () => calculateOrderTotal(items),  
    /quantity/  
  );  
});
```

## Что важно

- Negative test ожидает error.
- Если функция не выбросит error, test должен упасть.
- Для этого сама business function должна явно валидировать quantity.

# 1 Write test

## Написать тест



Сначала пишем тест, который описывает ожидаемое поведение.

```
import { test } from 'node:test';
import { strict as assert } from 'node:assert';
import { calculateOrderTotal } from './order.js';

test('total with discount', () => {
  const items = [
    { price: 100, quantity: 2 }
  ];
  const discount = 10;
  const result = calculateOrderTotal(items, discount);
  assert.equal(result, 180); // ожидаем 180
});
```

# 2 Run tests

## Запустить тесты



Запускаем тесты и проверяем результат. Тест падает, потому что функция ещё не реализована или содержит ошибку.

```
PS C:\project> npm test

> project@1.0.0 test
> node --test

✗ total with discount (24.1ms)
AssertionError [ERR_ASSERTION]:
Expected values to be strictly equal:

0 !== 180
```

# 6 Refactor

## Безопасно улучшить код



Улучшаем код: делаем его чище, быстрее, выносим повторения, добавляем проверки и т.д. Тесты гарантируют, что поведение не сломалось.

```
order.js (после рефакторинга)

export function calculateOrderTotal(
  items, discount = 0) {
  const subtotal = items.reduce(
    (sum, { price, quantity }) =>
      sum + price * quantity,
    0
  );
  const discountAmount = subtotal * (discount / 100);
  return Math.round(subtotal - discountAmount);
}
```

# 5 Pass

## Получить pass



Запускаем тесты снова. Теперь тест проходит.

```
PS C:\project> npm test

> project@1.0.0 test
> node --test

✓ total with discount (3.4ms)

Test run completed.
1 test passed (1)
0 failed (0)
```



# 3 Fail

## Увидеть failure



Тест не прошёл. Анализируем сообщение об ошибке, чтобы понять, что нужно исправить.

Сообщение об ошибке подсказывает:

- какой тест упал
- какое было ожидаемое значение
- какое значение получилось
- где находится проблема

# 4 Fix

## Реализовать или исправить код



Пишем или исправляем реализацию функции, чтобы тест проходил.

```
order.js

export function calculateOrderTotal(
  items, discount = 0) {
  const subtotal = items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );
  const discountAmount = subtotal * (discount / 100);
  return Math.round(subtotal - discountAmount);
}
```

# Checklist хорошего backend test



Проверяет конкретное поведение



Имеет понятное имя



Использует AAA



Имеет явный expected result



Не зависит от порядка других tests



Не использует production data



Проверяет boundary/negative cases



Быстро запускается на своём уровне



Падает по понятной причине



Можно многократно запускать одной командой

# Контрольные вопросы

1

1. Что такое regression?

2

2. Чем automated test отличается от manual testing?

3

3. Что обычно проверяет unit test?

4

4. Что обычно проверяет integration test?

5

5. Почему E2E tests обычно медленнее?

6

6. Что означает Arrange–Act–Assert?

7

7. Для чего нужна assertion?

8

8. Что такое flaky test?

9

9. Зачем test isolation?

10

10. Как запустить встроенный Node.js test runner из CLI?

# Итоги занятия

- 1 Автоматические tests уменьшают риск регрессий.
- 2 Unit, integration и E2E tests проверяют систему на разных уровнях.
- 3 Test pyramid предполагает много быстрых unit tests и меньше широких tests.
- 4 AAA разделяет подготовку, действие и проверку.
- 5 Assertions сравнивают actual и expected behavior.
- 6 Проверяйте happy path, invalid input и boundary values.
- 7 Хорошие tests изолированы и deterministic.
- 8 Production dependencies не должны бесконтрольно использоваться в unit tests.
- 9 Node.js имеет стабильный встроенный node:test runner.
- 10 Команда node --test позволяет запускать test suite автоматически.

# Следующая лекция

---

- Unit-тестирование с Jest
- Установка Jest
- describe() и test()
- expect() и matchers
- beforeEach/afterEach
- Mocks и spies
- Тестирование Service layer
- Изоляция Prisma dependency